Gemma 4 Rubber-Stamps Its Own Failing Patches

Sugam Panthi University of Southern Mississippi

github.com/Vein05/gemma4-paper-track Supplement: `rubber-stamp-supplement.pdf` Kaggle paper track

Abstract

To determine when a patch is complete, many coding agents ask the model that authored the patch. Self-evaluation of a patch works best if the model evaluates its own patches as accurately as it evaluates others' patches. We evaluate how well Gemma 4 31B self-evaluates its patches, mainly on the SWE-bench Verified dataset. Nine open models rate the likelihood that each patch will pass hidden tests. The judges are not informed as to which model authored the patch and are compared to outside judges from other model families on the same patch.

We find that Gemma places the highest score on 52 of 85 of its own failing patches, while outside judges place the same patches lower. The same holds in another team's Gemma run on SWE-bench Lite and on contest programs Gemma wrote without an agent. We also find that Gemma gives little additional trust to its passing patches and a varied amount of trust to the failing patches of other models. We detect no comparable effect in five other models on their own failing patches. Gemma ties many patches at the top score, so its score cannot select which patches to submit. In all, Gemma 4 rubber-stamps its own mistakes, so the agent built on it needs a second signal to determine which patches to submit.

1 Introduction

At the end of each task, coding agents are asked to make a decision to either submit the patch, reattempt the task, or hand the task off to a human. Many agents make this decision by asking the model that generated the patch whether it is correct. We evaluate this check for the Gemma 4 31B model used by the Gemma 4 Developer Agent Paper Track on Kaggle. Each judge is presented with a problem and one patch, without being told its author, and asked to score the chance that the patch passes hidden tests on a scale of 0 to 100. A model's own patches may simply be more difficult to judge, so we compare the results with outside judges, who are models from other families that are asked to score the same patches.

In the first instance, we study the publicly released Gemma patches for 50 issues, which were generated by four different agent programs (harnesses). For other models, we use a single harness (mini-SWE-agent), making the harness different only for Gemma. To remove this difference, we run Gemma 4 31B in mini-SWE-agent ourselves for the same 50 issues, and also for 193 held-out issues.

We find that Gemma gives the highest score to most of its own failing patches, while outside judges score the same patches lower. We call this rubber-stamping. Previous work has shown that neither Gemma 3 12B nor Gemma 3 27B shows self-preference when judging LiveCodeBench programs (Pombal et al., 2026). Gemma ranks its own patches a little less accurately than outside judges do, but this gap shrinks when Gemma's patches come from the same harness as the other models', so we treat it as weaker evidence (Section 3.3). Five other models show no ranking gap and little leniency on their own patches (Section 3.4, exploratory).

We make the following contributions:

- **Gemma 4 31B rubber-stamps its own failing patches.** On the held-out issues, Gemma gives 100 to 52 of

its 85 failing patches, and outside judges score the same patches lower. This share is 50 to 63% on every data set we test, including another team's run and programs written without an agent (Section 3.1).
- **Gemma's own run tells its patches apart.** Gemma gives 100 to most of its patches, so its score cannot tell which ones to submit. Its agent run can. Among the tied patches, those whose run took fewer model calls pass more often. Submitting the top quarter by this rule raises the share that pass from 61% to 78%, at no extra cost. A second judge, GLM-4.5, raises it to 84% (Section 3.2).

2 Setup

Patch sets. All issues come from SWE-bench Verified (Jimenez et al., 2023). We call the model that wrote a patch its writer. Other writers' patches are public submissions (SWE-bench experiments) made with mini-SWE-agent by GLM-4.5, gpt-oss-120b, Kimi-K2, Llama-4-Maverick, Qwen3-Coder and, on the Mini issues, Devstral, with their official labels. We removed test files from every patch and dropped patches over 30,000 characters.

Patch set	Issues	Gemma's patches	Scored by all nine judges (Gemma's / others')
Mini issues, mixed harness	50 (django, sphinx)	public runs in four harnesses	150 / 183
Mini issues, matched harness	the same 50	ours, mini-SWE-agent	42 / 183
Held-out issues	193, from 11 repositories	ours, mini-SWE-agent	180 / 777

The Mini issues are SWE-bench Verified Mini (MariusHobbhahn). Their mixed-harness Gemma patches come from public third-party runs (n-pelleriti) in SWE-agent (Yang et al., 2024), openclaw, opencode and zeroclaw. The held-out issues are a fixed random sample of 200 SWE-bench Verified issues outside the Mini issues. In our own runs, the 4-bit competition checkpoint of Gemma 4 31B made one attempt per issue in mini-SWE-agent v1.7.0 (SWE-agent), with its default settings and temperature 0. We graded these patches with SWE-bench's grading code and kept an issue only if its reference patch passes in our environment, which dropped 7 of the 200 held-out issues. Gemma solved 98 of the 193 issues we kept.

Judges. Nine models judged every patch: Gemma 4 31B, gpt-oss-20b, Qwen3.5-9B, GLM-4.7-Flash, and the five writers above other than Devstral. Calls went through OpenRouter at temperature 0 with reasoning switched off, except for gpt-oss, which always reasons and ran at low effort. We asked a judge once more, with a larger output limit, when its answer had no score (a re-ask). All analyses use the patches that all nine judges scored, which leaves Gemma's patches on 180 of the 193 held-out issues (Supplement, Sections B and C). As an exploratory check, we also report the held-out results without the patches that needed a re-ask.

Measures. We use four measures, with 95% intervals from 2,000 bootstrap resamples of issues.

- AUROC: the chance that a judge scores a random passing patch above a random failing one (ties count half).
- Self-judging gap: how much a model's AUROC drops from other writers' patches to its own, minus the mean drop of outside judges on the same patches.
- Leniency: a judge's score on a patch minus the outside judges' mean score on that patch. We report how much higher a model's mean leniency is on its own failing patches than on other writers' failing patches.
- Rubber-stamp rate: the share of a judge's own failing patches that it scores 100.

Plans. We committed a written plan for every test before computing its data. For our own patch sets, we changed two things after the plan but before reading any results. We added the leniency measure, and we raised the re-ask limit from 3,000 to 8,000 tokens because many answers ran out before the score. We mark as exploratory every analysis chosen after we saw a result.

Two more data sets. We also used two sources outside SWE-bench Verified (Supplement, Section K). Another team ran Gemma 4 31B on the 300 SWE-bench Lite issues, in mini-SWE-agent 2.2.1 at full precision with thinking on, and graded the patches. On LiveCodeBench, our 4-bit checkpoint wrote one program per contest problem, with no agent. Other writers' patches and programs are public submissions. Credit limits cut the judges to seven and six, before any score was read.

3 Results

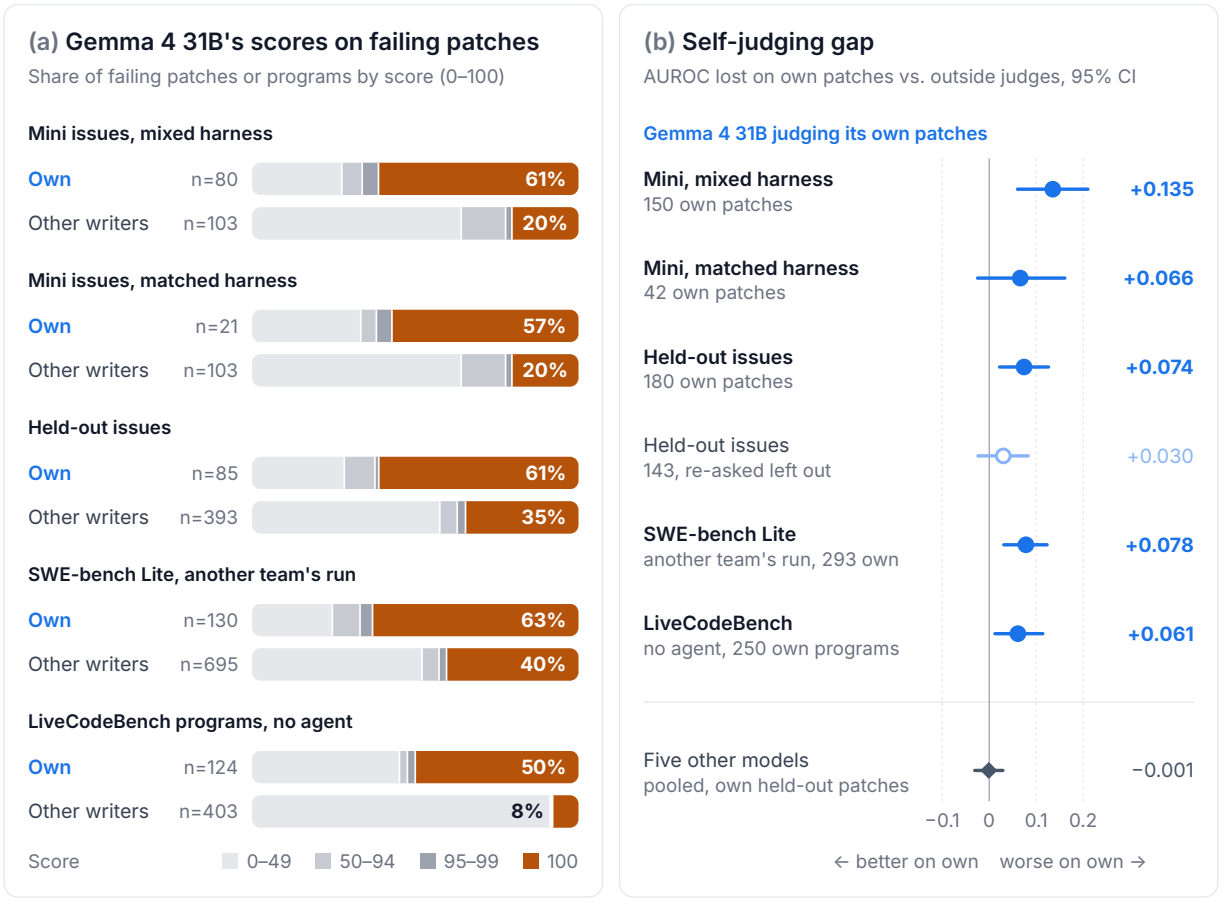


Figure 1. (a) Gemma 4 31B's scores on failing patches, its own and other writers', per data set. (b) Gemma's self-judging gap on each data set, with 95% intervals. The open circle leaves out every held-out patch that needed a re-ask. The diamond is the pooled gap of five other models judging their own patches.

3.1 Gemma rubber-stamps its own failing patches

Data set	Own failing patches scored 100	Others' failing patches scored 100	Leniency, own minus others' failing [95%]
Mini issues, mixed harness	61% (49 of 80)	20% (21 of 103)	+20.9 [11.5, 30.2]
Mini issues, matched harness	57% (12 of 21)	20% (21 of 103)	+11.7 [−1.4, 24.2]
Held-out issues	61% (52 of 85)	35% (136 of 393)	+16.6 [10.3, 23.1]
SWE-bench Lite, another team's run	63% (82 of 130)	40% (281 of 695)	+17.8 [12.1, 23.5]
LiveCodeBench programs, no agent	50% (62 of 124)	8% (32 of 403)	+23.0 [16.6, 29.9]

Gemma's rubber-stamp rate is 50 to 63% on every data set. It gives 100 to only 8 to 40% of other writers' failing patches. Some outside judges also give Gemma's failing patches 100 more often than other writers', so part of this difference may come from the patches. Leniency removes it by comparing judges on the same patch.

The pooled rate hides differences between writers. Gemma also gives 100 to 61% of Kimi-K2's failing held-out patches and to 68% of Claude 4 Sonnet's failing Lite patches (exploratory, Supplement, Section K). Its leniency on these two writers' failing patches is within the margin of error of its leniency on its own. Both writers solve more than half their issues, and Gemma is harsh on writers that solve under a third.

On the held-out issues, Gemma scores its own failing patches 14.1 points above the outside judges and other writers' failing patches 2.6 points below them. On passing patches, the difference is only +3.4 [−2.0, 8.0]. The held-out leniency is still +11.1 [3.8, 18.2] without the patches that needed a re-ask. On the 21 matched-harness failing patches the interval includes zero.

Three caveats apply to LiveCodeBench (Supplement, Section K). The other writers solve far fewer of the judged problems than Gemma. About 3 in 20 failing programs are grading errors. We cannot rule out that Gemma saw the problems in training.

Gemma rarely sees the defect that outside judges see. On matplotlib-24570, it scores its own failing patch 100, calling it "a simple swap" that "correctly implements the expected behavior". GLM-4.5 scores the same patch 20 because it "breaks VPacker by reversing left and right", and the hidden tests for left and right alignment fail. We coded Gemma's answers on the 31 own failing held-out patches that it scored 100 and at least two outside judges scored 50 or less (exploratory). Two coders, both Claude models working apart, agree that Gemma never mentions those judges' main defect in 23 of them and names it in 3. They disagree on the other 5 (Supplement, Section D).

Thinking does not stop the rubber-stamping. On the mixed-harness Mini patches, with thinking on, Gemma still gives 100 to 62% of its own failing patches, against 61% with thinking off (Supplement, Section D).

3.2 Breaking the tie at 100 picks better patches to submit

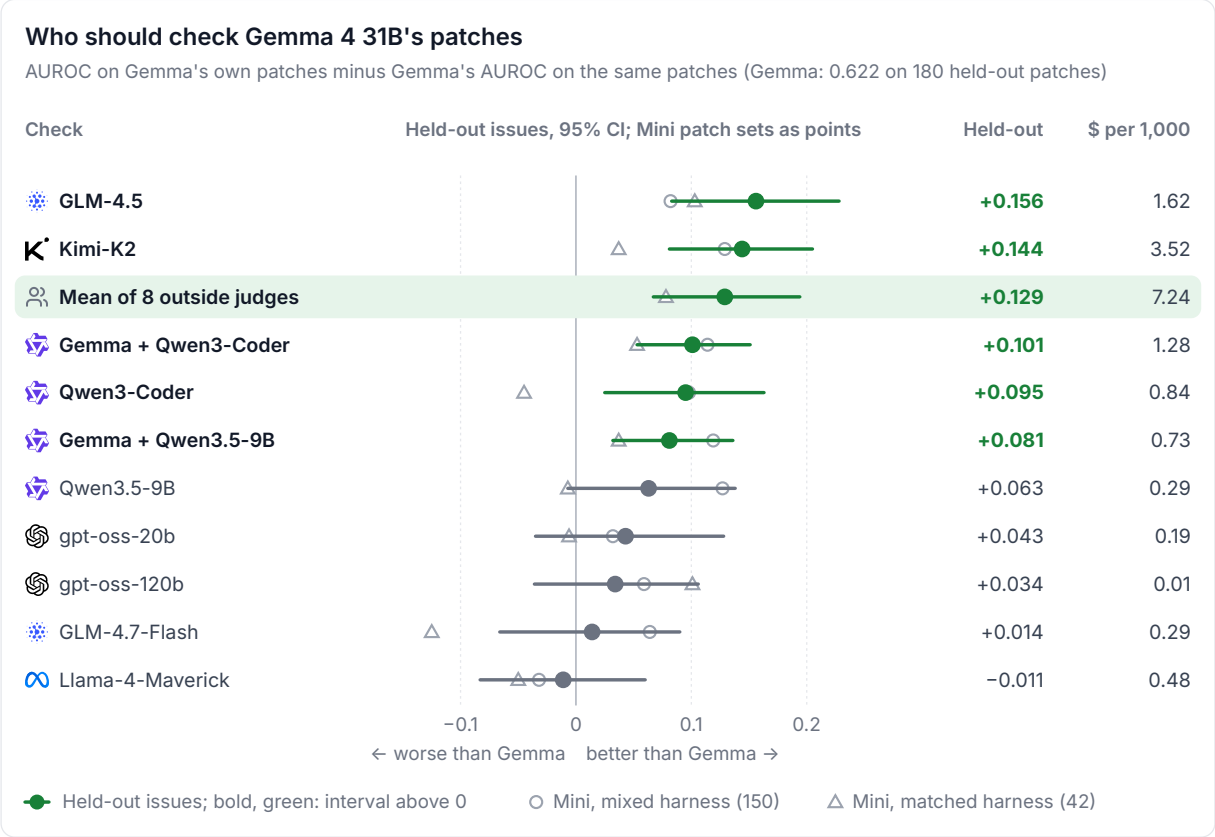


Figure 2. Each check's AUROC on Gemma's own patches minus Gemma's AUROC on the same patches. Lines are 95% intervals on the 180 held-out patches; hollow points are the two Mini patch sets. "Gemma + X" averages the two judges' ranks of each patch. Cost is what OpenRouter charged per 1,000 patches, in US dollars (gpt-oss-120b's provider reported almost zero cost).

We look at an agent that submits some of its patches without review and sends the rest to a person. The table below shows the share of held-out patches that pass when the agent submits the quarter that its check ranks highest. Without any check, 53% pass (95 of 180).

Check	Submit top quarter (45 patches): share passing, gain over Gemma's score in points [95%]
Gemma's own score	61%
Gemma, ties broken by fewer model calls	78%, +16.9 [5.7, 26.8]
Gemma + Qwen3.5-9B	76%, +14.9 [6.7, 23.3]
Kimi-K2	77%, +16.1 [9.6, 22.5]
GLM-4.5	84%, +22.7 [12.7, 32.0]

Gemma's score alone cannot lift the share passing above 61%. Gemma gives 100 to 133 of its 180 patches, and 81 of those pass (61%). Its top quarter is a random draw from that tied group. A second judge breaks the tie. GLM-4.5 and Kimi-K2 rank Gemma's patches better than Gemma does, by 0.16 and 0.14 in AUROC (Figure 2). Qwen3.5-9B alone is within the margin of error. Its average with Gemma's ranks gives a clear gain, at two cheap calls per patch.

We chose the three second-judge checks after seeing their AUROCs, so this comparison is exploratory. Over all 16 checks we ran, each outside judge alone and averaged with Gemma, the median top quarter passes 73% (Supplement, Section H). We also cannot tell whether the outside judges help because they come from another family or because they judge better in general.

Gemma's own agent run also breaks the tie, with no extra call. Among the patches Gemma scores 100, those whose run took fewer model calls pass more often. We chose this tie-break on the Mini issues, from three candidates, before testing it on the held-out issues, where it lifts the top quarter to 78%. This is within the margin of error of GLM-4.5, whose top quarter passes 5.9 points more [−3.6, 16.4] (exploratory). Run length may partly stand for how hard the issue is (Supplement, Section I).

Some patches go out without review, such as submissions and pull requests. For an agent built on Gemma, we advise ranking these with a second model, or at least breaking Gemma's ties by fewer model calls. Our data cannot tell whether a second judge also helps choose among attempts at one issue (Supplement, Section D).

3.3 Gemma ranks its own patches a little less accurately

Patch set	Gemma's AUROC, own patches	Gemma's AUROC, other writers' patches	Outside judges' drop	Self-judging gap [95%]
Mini issues, mixed harness	0.636	0.833	0.063	+0.135 [0.060, 0.210]
Mini issues, matched harness	0.696	0.833	0.071	+0.066 [−0.025, 0.161]
Held-out issues	0.622	0.751	0.055	+0.074 [0.022, 0.126]

Rubber-stamping should also hurt Gemma's ranking of its own patches, since a tie at 100 hides which of them fail. On the same 50 issues, the gap was about half as large when Gemma's patches came from the matched harness, though the two intervals overlap. The outside judges' drop stayed about the same, so the smaller gap comes from Gemma's own drop.

On the held-out issues, the gap meets the rule we fixed in advance for a replication (lower bound above zero and at least half the first estimate), but with little margin. It also depends on the larger re-ask limit. The gap is +0.030 [−0.024, 0.083] without the patches that needed a re-ask. On SWE-bench Lite and LiveCodeBench, the gap is above zero with or without re-asks (Supplement, Section K).

3.4 Five other models show no ranking gap and little leniency

On the held-out issues, all nine judges also scored the five other writers' patches, so each of these models judged its own patches, written in the same harness as Gemma's. Each gives 100 to at most 33% of its own failing patches, and at most 3 points more often than to other writers' failing patches. Their pooled gap is −0.001 [−0.031, 0.029] (Figure 1b), and their pooled leniency is +0.4 [−2.5, 3.4]. Gemma's leniency exceeds theirs by 16.3 [8.9, 23.4] points. No single model shows a gap or leniency once we correct for testing five models. This analysis is exploratory (Supplement, Section E).

4 Related work

Judges that favor their own outputs have been studied mostly on text and short programs. Panickssery et al. (2024) link the strength of this self-preference to how well a model recognizes its own summaries. Roytburg et al. (2026) compare a judge's own failing answers with other models' failing answers to the same question, and much of the measured self-preference goes away. Our leniency measure also compares judges

on the same patch. Chen et al. (2025) find that stronger judges more often prefer their own wrong answer to a correct one.

Two studies are closest to ours. Pombal et al. (2026) check programs from twelve models against unit-test rubrics and find that judges favor their own outputs even with objective rubrics. On LiveCodeBench, Gemma 3 12B and 27B do not: their ratios are 0.97 and 1.03, where 1 means no self-preference. We find Gemma 4 31B lenient on its own failing LiveCodeBench programs, with another measure. Khullar et al. (2026) find that a model rates a SWE-bench patch as more correct when it is framed as the model's own earlier answer. They report this for one unnamed model and do not compare it with other writers' patches. In our setting, the patches come from full agent runs, the judge sees no authorship or framing cue, and we measure ranking accuracy against hidden tests with outside judges as the control.

SWE-Gym (Pan et al., 2024) and R2E-Gym (Jain et al., 2025) fine-tune a verifier that reads an agent's run and patch, and use it to choose one of several attempts at an issue. Our second judge is an off-the-shelf model that decides which patches go out without review.

5 Limitations

Our finding is about Gemma 4 31B on Python code. We do not know what causes rubber-stamping, or whether Gemma recognizes its own patches or shares the mistakes in them. On the mixed-harness patches, neither patch size nor unsolved issues explain it (Supplement, Section D). Gemma as judge is the hosted model, served at a precision we did not record. With the 4-bit competition checkpoint that wrote the patches as judge, we detect no difference from the hosted model (Supplement, Section A).

We graded Gemma's patches ourselves, and other writers' patches keep their official labels. In our environment, 26 of 30 other-writer patches got their official label. All four misses were passes graded as failures, from environment faults or patches that no longer apply. The same error on Gemma's patches would put real passes among its failing patches and inflate the rubber-stamp rate. We guarded against both causes: each issue we kept has a reference patch that passes in our environment, and Gemma wrote its patches on the same images.

6 Data, code and supplement

https://github.com/Vein05/gemma4-paper-track

References

Wei-Lin Chen, Zhepei Wei, Xinyu Zhu, Shi Feng and Yu Meng. 2025. *Do LLM Evaluators Prefer Themselves for a Reason?* https://arxiv.org/abs/2504.03846.

Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen and Ion Stoica. 2025. *R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents*. https://arxiv.org/abs/2504.07164.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press and Karthik Narasimhan. 2023. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770. ICLR 2024 (arXiv comment, https://openreview.net/forum?id=VTF8yNQM66).

Dipika Khullar, Jack Hopkins, Rowan Wang and Fabien Roger. 2026. *Self-Attribution Bias: When AI Monitors Go Easy on Themselves*. https://arxiv.org/abs/2603.04582.

MariusHobbhahn. *swe-bench-verified-mini*. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini. Hugging Face dataset, revision b316c349, accessed 2026-10-05.

n-pelleriti. *AlphaDiana Gemma 4 agent-run datasets*. https://huggingface.co/n-pelleriti. Six Hugging Face datasets; accessed 2026-10-05.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr and Yizhe Zhang. 2024. *Training Software Engineering Agents and Verifiers with SWE-Gym*. https://arxiv.org/abs/2412.21139. Accepted at ICML 2025.

Arjun Panickssery, Samuel R. Bowman and Shi Feng. 2024. *LLM Evaluators Recognize and Favor Their Own Generations*. https://arxiv.org/abs/2404.13076.

José Pombal, Ricardo Rei and André F. T. Martins. 2026. *Self-Preference Bias in Rubric-Based Evaluation of Large Language Models*. https://arxiv.org/abs/2604.06996.

Dani Roytburg, Matthew Bozoukov, Matthew Nguyen, Jou Barzdukas, Mackenzie Puig-Hall and Narmeen Oozeer. 2026. *Are LLM Evaluators Really Narcissists? Sanity Checking Self-Preference Evaluations*. https://arxiv.org/abs/2601.22548. ICML 2026 Main.

SWE-agent. *mini-SWE-agent*. https://github.com/SWE-agent/mini-swe-agent. GitHub repository; accessed 2026-10-05.

SWE-bench. *SWE-bench experiments*. https://github.com/SWE-bench/experiments. GitHub repository, patches from the swe-bench-submissions bucket, revision 40f164d5, accessed 2026-10-05.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan and Ofir Press. 2024. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. https://arxiv.org/abs/2405.15793.